GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	Sockets				
NÚMERO DE PRÁCTICA:	**03**	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026A
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		**4**
FECHA INICIO:	**27/04/2026**	**FECHA FIN:**	**01/05/2026**	**DURACIÓN:**	**2 horas**
RECURSOS A UTILIZAR: VSCode, Netbeans, Eclipse, otros como: Python, C++,C#,etc.					
DOCENTE(s): Mg. Maribel Molina Barriga					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Aplica el uso de sockets en java para la solución de sistemas de distribuidos	
TEMAS: • Los Sockets primitivos • Sockets primitivos en Java. Comprender el protocolo TCP, UDP. • Implementar un proceso de comunicación	
COMPETENCIAS	C.a Aplica de forma transformadora conocimientos de matemática, computación eingeniería como herramienta para evaluar, sintetizar y mostrar información como fundamento de sus ideas y perspectivas para la resolución de problemas.

CONTENIDO DE LA GUÍA

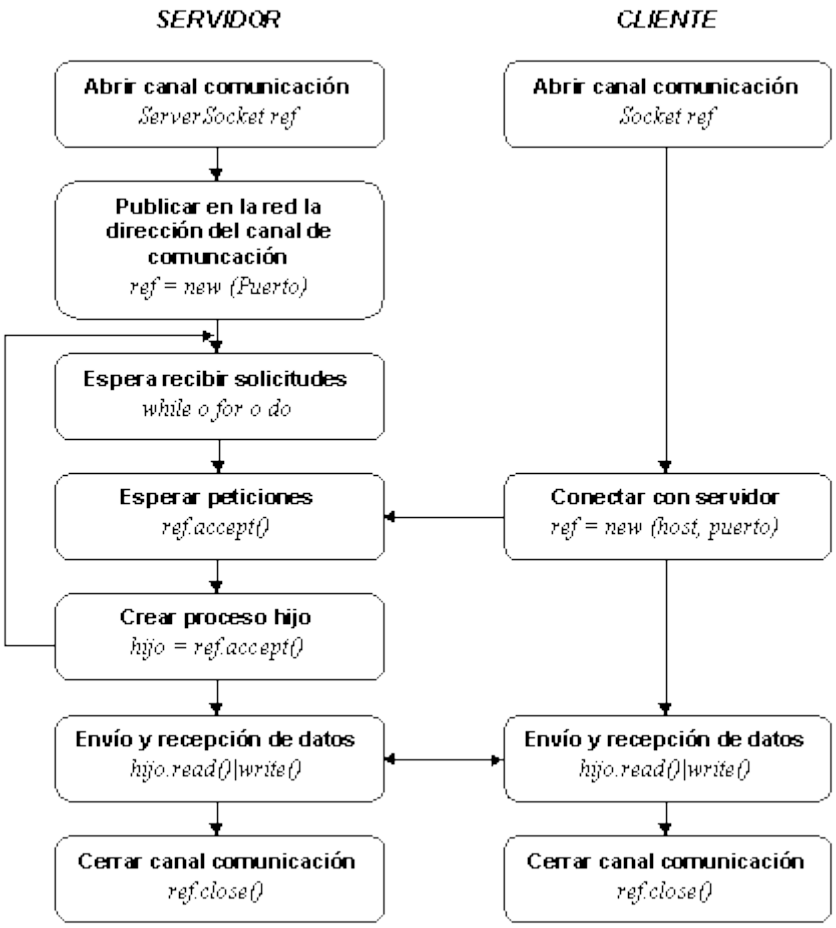
I. MARCO CONCEPTUAL

1. Los Sockets

- Son un sistema de comunicación entre procesos de diferentes máquinas de una red. Más exactamente, un socket es un punto de comunicación por el cual un proceso puede emitir o recibir información.
- Fueron popularizados por Berckley Software Distribution, de la universidad norteamericana de Berkley. Los sockets han de ser capaces de utilizar el protocolo de streams TCP (Transfer Control Protocol) y el de datagramas UDP (User Datagram Protocol).
- Utilizan una serie de primitivas para establecer el punto de comunicación, para conectarse a una máquina remota en un determinado puerto que esté disponible, para escuchar en él, para leer o escribir

y publicar información en él, y finalmente para desconectarse.

- Con todas las primitivas se puede crear un sistema de diálogo muy completo



Funcionamiento de una conexión socket

2. Funcionamiento genérico

Normalmente, un servidor se ejecuta sobre una computadora específica y tiene un socket que responde en un puerto específico. El servidor únicamente espera, escuchando a través del socket a que un cliente haga una petición.

En el lado del cliente: el cliente conoce el nombre de host de la máquina en la cual e el servidor se encuentra ejecutando y el número de puerto en el cual el servidor está conectado. Para realizar una petición de conexión , el cliente intenta encontrar al servidor en la máquina servidora en el puerto especificado.

Si todo va bien, el servidor acepta la conexión. Además de aceptar, el servidor obtiene un nuevo socket sobre un puerto diferente. Esto se debe a que necesita un nuevo socket (y , en consecuencia, un numero de puerto diferente) para seguir atendiendo al socket original para peticiones de conexión mientras atiende las necesidades del cliente que se conectó.

Por la parte del cliente, si la conexión es aceptada, un socket se crea de forma satisfactoria y puede usarlo para comunicarse con el servidor. Es importante darse cuenta que el socket en el cliente no está utilizando el número de puerto usado para realizar la petición al servidor. En lugar de éste, el cliente asigna un número de puerto local a la máquina en la cual está siendo ejecutado. Ahora el cliente y el servidor pueden comunicarse escribiendo o leyendo en o desde sus respectivos sockets.

3. Sockets en Java

Java incluye la librería java.net para la utilización de sockets, tanto TCP como UDP.
El paquete java.net de la plataforma Java proporciona una clase Socket, la cual implementa una de las partes de la comunicación bidireccional entre un programa Java y otro programa en la red. La clase Socket se sitúa en la parte más alta de una implementación dependiente de la plataforma, ocultando los detalles de cualquier sistema particular al programa Java.

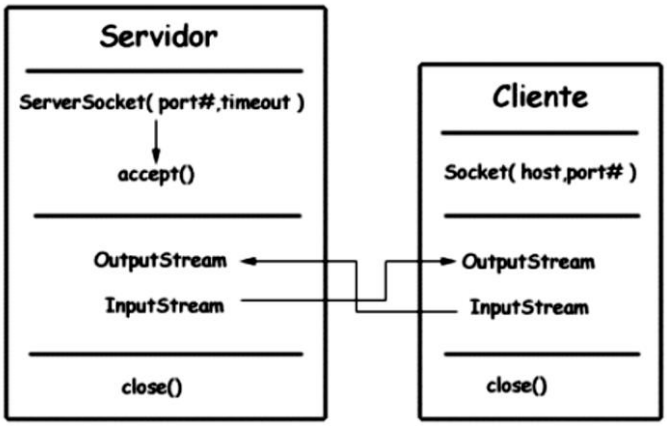
Usando la clase java.net, Socket en lugar de utilizar código nativo de la plataforma, los programas Java pueden comunicarse a través de la red de una forma totalmente independiente de la plataforma.
De forma adicional, java.net incluye la clase Server Socket, la cual implementa un socket el cual los servidores pueden utilizar para escuchar y aceptar peticiones de conexión de clientes.

Nuestro objetivo será conocer cómo utilizar las clases Socket y Server Socket. Por otra parte, si intentamos conectar a través de la Web, la clase URL y clases relacionadas (URL Connection, URL Encoder) son probablemente más apropiadas que las clases de sockets. Las clases URL no son más que una conexión a un nivel más alto a la Web y utilizan como parte de su implementación interna los sockets.

4. Modelo de comunicaciones con Java

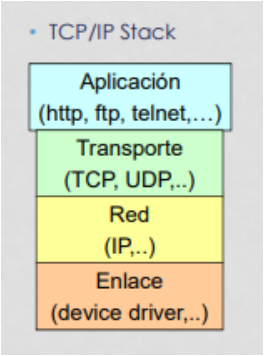
El modelo de sockets más simple es:

- El servidor establece un puerto y espera durante un cierto tiempo (time out segundos), a que el cliente establezca la conexión. Cuando el cliente solicite una conexión, el servidor abrirá la conexión socket con el método accept().
- El cliente establece una conexión con la máquina host a través del puerto que se designe en puerto#
- El cliente y el servidor se comunican con manejadores InputStream y OutputStream



5. Trabajo en red básico

Los ordenadores que se ejecutan en Internet se comunican unos con otros utilizando los protocolos TCP y UDP, que son protocolos de 4 capas.



TCP

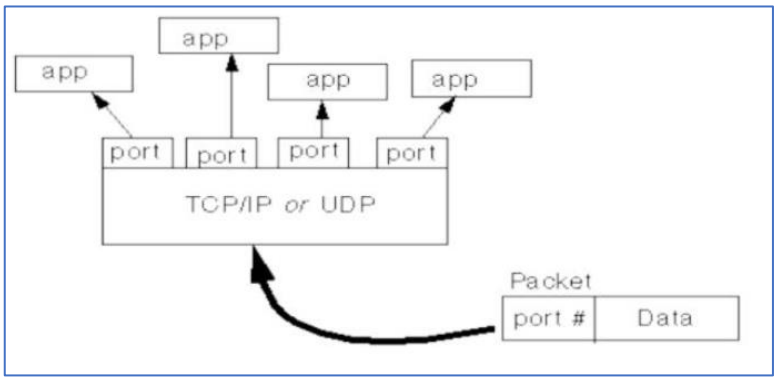
Es un protocolo basado en conexión que proporciona un flujo fiable de datos entre dos ordenadores.
Las aplicaciones que requieren fiabilidad, canales punto a punto para comunicarse, utilizan TCP para ello.
Hyper Text Transfer Protocol (HTTP), File Transfer Protocol (ftp), y Telnet (telnet) son ejemplos de aplicaciones que requieren un canal de comunicación fiable.

UDP

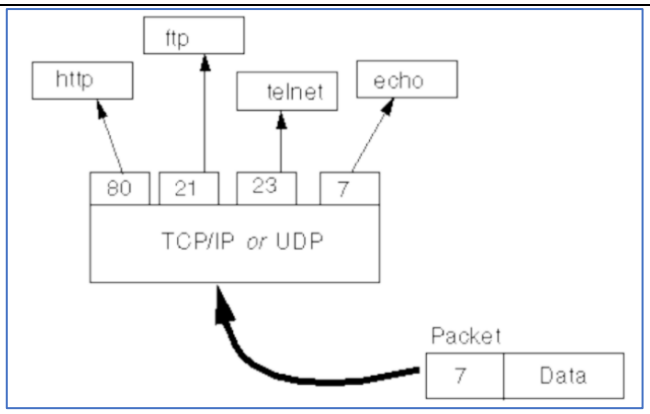
Es un protocolo que envía paquetes de datos independientes, llamados datagramas desde un ordenador a otro sin garantías sobre su llegada. UDP no está basado en la conexión como TCP.
El protocolo UDP proporciona una comunicación no garantizada entre dos aplicaciones en la Red. UDP envía estos paquetes de datos o datagramas de una aplicación a la otra. Enviar datagramas es como enviar una carta a través del servicio de correos: el orden de envío no es importante y no está garantizado, y cada mensaje es independiente de los otros.

PUERTOS

Los protocolos TCP y UDP utilizan puertos para dirigir los datos de entrada a los procesos particulares que se están ejecutando en un ordenador.



Los números de puertos tienen un rango de 0 a 65535 (porque los puertos están representados por un número de 16 bits). Los puertos entre los números 0 - 1023 están restringidos -- están reservados para servicios bien conocidos como HTTP, FTP y otros servicios del sistema. Tus aplicaciones no deberían intentar unirse a estos puertos. Los puertos que están reservados para los servicios bien conocidos como HTTP y FTP son llamados puertos conocidos.



6. Las clases Socket

Son utilizadas para representar conexiones entre un programa cliente y otro programa servidor.
El paquete java.net proporciona dos clases -- Socket y ServerSocket -- que implementan los lados del cliente y del servidor de una conexión, respectivamente.

TCP proporciona un canal de comunicación fiable punto a punto, lo que utilizan para comunicarse las aplicaciones cliente-servidor en Internet. Las clases Socket y ServerSocket del paquete java.net proporcionan un canal de comunicación independiente del sistema utilizando TCP.

Conectores "para clientes"

Los conectores (sockets), o los conectores TCP/IP en concreto, se utilizan para implementar conexiones basadas en flujo, punto a punto, bidireccionales y fiables entre nodos de Internet. Se puede utilizar un conector para conectar el sistema de E/S de Java a otros programas que pueden residir en la máquina local o en cualquier otra máquina de Internet. La clase Socket, a diferencia de DatagramSocket, implementa una conexión continua muy fiable entre el cliente y el servidor.
Al crear un objeto conector también se establece la conexión entre direcciones de Internet. No hay métodos ni constructores que muestren explícitamente los detalles del establecimiento de la conexión del cliente. Se puedan utilizar dos constructores para crear conectores:

- ***Socket(String nodo, int puerto***) crea un conector que conecta el nodo local con el nodo y puerto nombrados.
- ***Socket(InetAddress dirección, int puerto)*** crea un conector utilizando un objeto InetAddress ya existente y un puerto.

En un conector se puede examinar en cualquier momento la información de dirección y puerto asociada con él utilizando los métodos siguientes:

- ***getInetAddress()*** devuelve la InetAddress asociada con el objeto Socket.
- ***getPort()*** devuelve el puerto remoto al que está conectado este objeto Socket.
- ***getLocalPort()*** devuelve el puerto local al que está conectado este objeto Socket.

Cuando se ha creado el objeto Socket, también puede ser examinado para acceder a los flujos de entrada y salida asociados con él. Todos estos métodos pueden lanzar una IOException si se han invalidado los conectores debido a una pérdida de conexión en la red. Estos flujos se utilizan exactamente igual que los flujos de E/S que hemos visto en el capítulo anterior para enviar y recibir datos:

- ***getInputStream()*** devuelve el InputStream asociado con este conector.
- ***getOutputStream()*** devuelve el OutputStream asociado con este conector.
- ***close()*** cierra el InputStream y el OutputStream.

Conectores "para servidores"

Los ServerSockets se deben utilizar para crear servidores de Internet. Estos servidores no son necesariamente máquinas, de hecho, son programas que están esperando a que programas cliente locales

o remotos se conecten a ellos en puertos públicos. Los ServerSockets son bastante diferentes de los Sockets normales. Cuando se crea un ServerSocket, se registrará en el sistema que tiene interés en conexiones de cliente. Tiene un método adicional, accept, que es una llamada que se bloquea ya que espera que un cliente inicie la comunicación y después devuelve un Socket normal.

Los dos constructores de ServerSocket reflejan el número del puerto en el que se desean aceptar las conexiones y, opcionalmente, durante cuánto tiempo se desea esperar a que se deje de utilizar el puerto. Ambos constructores pueden lanzar una IOException bajo condiciones adversas. Estos son los dos prototipos:

- ***ServerSocket(int puerto)*** crea un conector de servidor en el puerto especificado.
- ***ServerSocket(int puerto, int número)*** crea un conector de servidor en el puerto especificado esperando número milisegundos si el puerto se está utilizando.

Funcionalmente, el método accept de un ServerSocket es una llamada que se bloquea y que espera que un cliente inicie la comunicación y después devuelve un Socket normal. Después, este conector se utiliza para la comunicación con el cliente.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Para comprender el funcionamiento de los sockets se presentará una aplicación que establece un pequeño diálogo entre un programa servidor y sus clientes, que intercambiarán cadenas de información.

a) Programa Cliente

El programa cliente se conecta a un servidor indicando el nombre de la máquina y el número puerto (tipo de servicio que solicita) en el que el servidor está instalado.

Una vez conectado, lee una cadena del servidor y la escribe en la pantalla:

```
import java.io.*;
import java.net.*;
public class Cliente {

        static final String HOST = "localhost";

          static final int PUERTO=5000;

          public Cliente( ) {

            try{

              Socket skCliente = new Socket( HOST , PUERTO );

              InputStream aux = skCliente.getInputStream();

              DataInputStream flujo = new DataInputStream( aux );

              System.out.println( flujo.readUTF() );

              skCliente.close();

            } catch( Exception e ) {

              System.out.println( e.getMessage() );

            }

          }

          public static void main( String[] arg ) {

            new Cliente();

          }

        }
```

- En primer lugar se crea el socket denominado skCliente, al que se le especifican el nombre de host (HOST) y el número de puerto (PORT) en este ejemplo constantes.

- Luego se asocia el flujo de datos de dicho socket (obtenido mediante getInputStream)), que es asociado a un flujo (flujo) DataInputStream de lectura secuencial. De dicho flujo capturamos una cadena (readUTF()), y la imprimimos por pantalla (System.out).
- El socket se cierra, una vez finalizadas las operaciones, mediante el método close().
- Debe observarse que se realiza una gestión de excepción para capturar los posibles fallos tanto de los flujos de datos como del socket.

b) Programa Servidor

El programa servidor se instala en un puerto determinado, a la espera de conexiones, a las que tratará mediante un segundo socket.

Cada vez que se presenta un cliente, le saluda con una frase "Hola cliente N".

Este servidor sólo atenderá hasta tres clientes, y después finalizará su ejecución, pero es habitual utilizar bucles infinitos (while(true)) en los servidores, para que atiendan llamadas continuamente.

Tras atender cuatro clientes, el servidor deja de ofrecer su servicio:

```
import java.io.*;
import java.net.*;
public class Servidor {
        static final int PUERTO=5000;

          public Servidor( ) {

            try {

              ServerSocket skServidor = new ServerSocket(PUERTO);

              System.out.println("Escucho el puerto " + PUERTO);

              for ( int numCli = 0; numCli < 3; numCli++ ) {

                Socket skCliente = skServidor.accept(); // Crea objeto

                System.out.println("Sirvo al cliente " + numCli);

                OutputStream aux = skCliente.getOutputStream();

                DataOutputStream flujo= new DataOutputStream( aux );

                flujo.writeUTF( "Hola cliente " + numCli );

                skCliente.close();

              }

              System.out.println("Demasiados clientes por hoy");

            } catch( Exception e ) {

              System.out.println( e.getMessage() );

            }

          }

          public static void main( String[] arg ) {

            new Servidor();

          }

}
```

- Utiliza un objeto de la clase ServerSocket (skServidor), que sirve para esperar las conexiones en un puerto determinado (PUERTO), y un objeto de la clase Socket (skCliente) que sirve para gestionar una conexión con cada cliente.

- Mediante un bucle for y la variable numCli se restringe el número de clientes a tres, con lo que cada vez que en el puerto de este servidor aparezca un cliente, se atiende y se incrementa el contador.
- Para atender a los clientes se utiliza la primitiva accept() de la clase ServerSocket, que es una rutina que crea un nuevo Socket (skCliente) para atender a un cliente que se ha conectado a ese servidor.
- Se asocia al socket creado (skCliente) un flujo (flujo) de salida DataOutputStream de escritura secuencial, en el que se escribe el mensaje a enviar al cliente.
- El tratamiento de las excepciones es muy reducido en nuestro ejemplo, tan solo se captura e imprime el mensaje que incluye la excepción mediante getMessage().

c) Ejecución

Aunque la ejecución de los sockets está diseñada para trabajar con ordenadores en red, en sistemas operativos multitarea (por ejemplo Windows y LINUX se puede probar el correcto funcionamiento de un programa de sockets en una misma máquina.

Para ellos se ha de colocar el servidor en una ventana de consola CMD o en tal caso si estamos en Windows ejecutar el Windows PowerShell, obteniendo lo siguiente:

>javac Servidor.java
>java Servidor
Escucho el puerto 5000
En otra ventana se lanza varias veces el programa cliente, obteniendo:
>javac Cliente.java
>java Cliente
Hola cliente 1
>java cliente
Hola cliente 2
…
connection refused: no further information
Mientras tanto en la ventana del servidor se ha impreso:
Sirvo al cliente 1
Sirvo al cliente 2
Sirvo al cliente …
Demasiados clientes por hoy

Obsérvese que tanto el cliente como el servidor pueden leer o escribir del socket. Los mecanismos de comunicación pueden ser refinados cambiando la implementación de los sockets, mediante la utilización de las clases abstractas que el paquete java.net provee.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

- Analizar, implementar, ejecutar y probar el código del ejercicio de Ejemplo de Sockets en Java, según el **Anexo 1.**
- Para luego realizar lo siguiente:
  - Evaluar resultados obtenidos.
  - Escriba un reporte sobre las tareas realizadas y resultados.
  - Escriba sus conclusiones

IV. CUESTIONARIO

1. ¿Por qué y en qué momento usar sockets TCP y sockets UDP?
2. En un socket TCP, ¿cuándo sabe el servidor que el cliente ha cerrado la conexión?

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
[3]Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.

[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN	
TÉCNICAS: *Problemas /Ejercicios propuestos / Preguntas formuladas / Resolución de casos*	**INSTRUMENTOS:** *Lista de cotejo*
CRITERIOS DE EVALUACIÓN • Identifica los programas con sockets en Java • Utiliza código de sockets en Java para aplicaciones específicas.	

Anexo 1

ChatMessage.java

```java
import java.io.*;
/*
 * This class defines the different type of messages that will be exchanged between the
 * Clients and the Server.
 * When talking from a Java Client to a Java Server a lot easier to pass Java objects, no
 * need to count bytes or to wait for a line feed at the end of the frame
 */

public class ChatMessage implements Serializable {

        // The different types of message sent by the Client
        // WHOISIN to receive the list of the users connected
        // MESSAGE an ordinary text message
        // LOGOUT to disconnect from the Server
        static final int WHOISIN = 0, MESSAGE = 1, LOGOUT = 2;
        private int type;
        private String message;

        // constructor
        ChatMessage(int type, String message) {
                this.type = type;
                this.message = message;
        }

        int getType() {
                return type;
        }

        String getMessage() {
                return message;
        }
}
```

Client.java

```java
import java.net.*;
import java.io.*;
import java.util.*;

//The Client that can be run as a console
public class Client  {

        // notification
        private String notif = " *** ";

        // for I/O
        private ObjectInputStream sInput;               // to read from the socket
        private ObjectOutputStream sOutput;             // to write on the socket
        private Socket socket;                                  // socket object

        private String server, username;// server and username
        private int port;                                       //port

        public String getUsername() {
                return username;
        }

        public void setUsername(String username) {
                this.username = username;
        }

        /*
         *  Constructor to set below things
         *  server: the server address
         *  port: the port number
         *  username: the username
         */

        Client(String server, int port, String username) {
                this.server = server;
                this.port = port;
                this.username = username;
        }

        /*
         * To start the chat
         */
        public boolean start() {
                // try to connect to the server
                try {
                        socket = new Socket(server, port);
                }
                // exception handler if it failed
                catch(Exception ec) {
                        display("Error connectiong to server:" + ec);
                        return false;
                }

                String msg = "Connection accepted " + socket.getInetAddress() + ":" + socket.getPort();
                display(msg);

                /* Creating both Data Stream */
                try
                {
                        sInput  = new ObjectInputStream(socket.getInputStream());
                        sOutput = new ObjectOutputStream(socket.getOutputStream());
                }
                catch (IOException eIO) {
                        display("Exception creating new Input/output Streams: " + eIO);
                        return false;
                }

                // creates the Thread to listen from the server
                new ListenFromServer().start();
                // Send our username to the server this is the only message that we
                // will send as a String. All other messages will be ChatMessage objects
                try
```

```
        {
                sOutput.writeObject(username);
        }
        catch (IOException eIO) {
                display("Exception doing login : " + eIO);
                disconnect();
                return false;
        }
        // success we inform the caller that it worked
        return true;
}

/*
 * To send a message to the console
 */
private void display(String msg) {

        System.out.println(msg);

}

/*
 * To send a message to the server
 */
void sendMessage(ChatMessage msg) {
        try {
                sOutput.writeObject(msg);
        }
        catch(IOException e) {
                display("Exception writing to server: " + e);
        }
}

/*
 * When something goes wrong
 * Close the Input/Output streams and disconnect
 */
private void disconnect() {
        try {
                if(sInput != null) sInput.close();
        }
        catch(Exception e) {}
        try {
                if(sOutput != null) sOutput.close();
        }
        catch(Exception e) {}
try{
                if(socket != null) socket.close();
        }
        catch(Exception e) {}

}
/*
 * To start the Client in console mode use one of the following command
 * > java Client
 * > java Client username
 * > java Client username portNumber
 * > java Client username portNumber serverAddress
 * at the console prompt
 * If the portNumber is not specified 1500 is used
 * If the serverAddress is not specified "localHost" is used
 * If the username is not specified "Anonymous" is used
 */
public static void main(String[] args) {
        // default values if not entered
        int portNumber = 1500;
        String serverAddress = "localhost";
        String userName = "Anonymous";
        Scanner scan = new Scanner(System.in);

        System.out.println("Enter the username: ");
        userName = scan.nextLine();
```

```
                // different case according to the length of the arguments.
                switch(args.length) {
                        case 3:
                                // for > javac Client username portNumber serverAddr
                                serverAddress = args[2];
                        case 2:
                                // for > javac Client username portNumber
                                try {
                                        portNumber = Integer.parseInt(args[1]);
                                }
                                catch(Exception e) {
                                        System.out.println("Invalid port number.");
                                        System.out.println("Usage is: > java Client [username] [portNumber]
[serverAddress]");
                                        return;
                                }
                        case 1:
                                // for > javac Client username
                                userName = args[0];
                        case 0:
                                // for > java Client
                                break;
                        // if number of arguments are invalid
                        default:
                                System.out.println("Usage is: > java Client [username] [portNumber]
[serverAddress]");
                        return;
                }
                // create the Client object
                Client client = new Client(serverAddress, portNumber, userName);
                // try to connect to the server and return if not connected
                if(!client.start())
                        return;

                System.out.println("\nHello.! Welcome to the chatroom.");
                System.out.println("Instructions:");
                System.out.println("1. Simply type the message to send broadcast to all active clients");
                System.out.println("2. Type '@username<space>yourmessage' without quotes to send message to
desired client");
                System.out.println("3. Type 'WHOISIN' without quotes to see list of active clients");
                System.out.println("4. Type 'LOGOUT' without quotes to logoff from server");

                // infinite loop to get the input from the user
                while(true) {
                        System.out.print("> ");
                        // read message from user
                        String msg = scan.nextLine();
                        // logout if message is LOGOUT
                        if(msg.equalsIgnoreCase("LOGOUT")) {
                                client.sendMessage(new ChatMessage(ChatMessage.LOGOUT, ""));
                                break;
                        }
                        // message to check who are present in chatroom
                        else if(msg.equalsIgnoreCase("WHOISIN")) {
                                client.sendMessage(new ChatMessage(ChatMessage.WHOISIN, ""));

                        }
                        // regular text message
                        else {
                                client.sendMessage(new ChatMessage(ChatMessage.MESSAGE, msg));
                        }
                }
                // close resource
                scan.close();
                // client completed its job. disconnect client.
                client.disconnect();
        }

        /*
         * a class that waits for the message from the server
         */
```

```
        class ListenFromServer extends Thread {

                public void run() {
                        while(true) {
                                try {
                                        // read the message form the input datastream
                                        String msg = (String) sInput.readObject();
                                        // print the message
                                        System.out.println(msg);
                                        System.out.print("> ");
                                }
                                catch(IOException e) {
                                        display(notif + "Server has closed the connection: " + e + notif);
                                        break;
                                }
                                catch(ClassNotFoundException e2) {
                                }
                        }
                }
        }
}
```

Server.java

```
import java.io.*;
import java.net.*;
import java.text.SimpleDateFormat;
import java.util.*;

// the server that can be run as a console
public class Server {
        // a unique ID for each connection
        private static int uniqueId;
        // an ArrayList to keep the list of the Client
        private ArrayList<ClientThread> al;
        // to display time
        private SimpleDateFormat sdf;
        // the port number to listen for connection
        private int port;
        // to check if server is running
        private boolean keepGoing;
        // notification
        private String notif = " *** ";

        //constructor that receive the port to listen to for connection as parameter

        public Server(int port) {
                // the port
                this.port = port;
                // to display hh:mm:ss
                sdf = new SimpleDateFormat("HH:mm:ss");
                // an ArrayList to keep the list of the Client
                al = new ArrayList<ClientThread>();
        }

        public void start() {
                keepGoing = true;
                //create socket server and wait for connection requests
                try
                {
                        // the socket used by the server
                        ServerSocket serverSocket = new ServerSocket(port);

                        // infinite loop to wait for connections ( till server is active )
                        while(keepGoing)
                        {
                                display("Server waiting for Clients on port " + port + ".");

                                // accept connection if requested from client
                                Socket socket = serverSocket.accept();
                                // break if server stoped
```

```
                        if(!keepGoing)
                                break;
                        // if client is connected, create its thread
                        ClientThread t = new ClientThread(socket);
                        //add this client to arraylist
                        al.add(t);

                        t.start();
                }
                // try to stop the server
                try {
                        serverSocket.close();
                        for(int i = 0; i < al.size(); ++i) {
                                ClientThread tc = al.get(i);
                                try {
                                // close all data streams and socket
                                tc.sInput.close();
                                tc.sOutput.close();
                                tc.socket.close();
                                }
                                catch(IOException ioE) {
                                }
                        }
                }
                catch(Exception e) {
                        display("Exception closing the server and clients: " + e);
                }
        }
        catch (IOException e) {
    String msg = sdf.format(new Date()) + " Exception on new ServerSocket: " + e + "\n";
                display(msg);
        }
}

// to stop the server
protected void stop() {
        keepGoing = false;
        try {
                new Socket("localhost", port);
        }
        catch(Exception e) {
        }
}

// Display an event to the console
private void display(String msg) {
        String time = sdf.format(new Date()) + " " + msg;
        System.out.println(time);
}

// to broadcast a message to all Clients
private synchronized boolean broadcast(String message) {
        // add timestamp to the message
        String time = sdf.format(new Date());

        // to check if message is private i.e. client to client message
        String[] w = message.split(" ",3);

        boolean isPrivate = false;
        if(w[1].charAt(0)=='@')
                isPrivate=true;


        // if private message, send message to mentioned username only
        if(isPrivate==true)
        {
                String tocheck=w[1].substring(1, w[1].length());

                message=w[0]+w[2];
                String messageLf = time + " " + message + "\n";
                boolean found=false;
                // we loop in reverse order to find the mentioned username
```

```
                        for(int y=al.size(); --y>=0;)
                        {
                                ClientThread ct1=al.get(y);
                                String check=ct1.getUsername();
                                if(check.equals(tocheck))
                                {
                                        // try to write to the Client if it fails remove it from the list
                                        if(!ct1.writeMsg(messageLf)) {
                                                al.remove(y);
                                                display("Disconnected Client " + ct1.username + " removed from
list.");
                                        }
                                        // username found and delivered the message
                                        found=true;
                                        break;
                                }



                        }
                        // mentioned user not found, return false
                        if(found!=true)
                        {
                                return false;
                        }
                }
                // if message is a broadcast message
                else
                {
                        String messageLf = time + " " + message + "\n";
                        // display message
                        System.out.print(messageLf);

                        // we loop in reverse order in case we would have to remove a Client
                        // because it has disconnected
                        for(int i = al.size(); --i >= 0;) {
                                ClientThread ct = al.get(i);
                                // try to write to the Client if it fails remove it from the list
                                if(!ct.writeMsg(messageLf)) {
                                        al.remove(i);
                                        display("Disconnected Client " + ct.username + " removed from list.");
                                }
                        }
                }
                return true;


        }

        // if client sent LOGOUT message to exit
        synchronized void remove(int id) {

                String disconnectedClient = "";
                // scan the array list until we found the Id
                for(int i = 0; i < al.size(); ++i) {
                        ClientThread ct = al.get(i);
                        // if found remove it
                        if(ct.id == id) {
                                disconnectedClient = ct.getUsername();
                                al.remove(i);
                                break;
                        }
                }
                broadcast(notif + disconnectedClient + " has left the chat room." + notif);
        }

        /*
         *  To run as a console application
         * > java Server
         * > java Server portNumber
         * If the port number is not specified 1500 is used
         */
```

```java
public static void main(String[] args) {
        // start server on port 1500 unless a PortNumber is specified
        int portNumber = 1500;
        switch(args.length) {
                case 1:
                        try {
                                portNumber = Integer.parseInt(args[0]);
                        }
                        catch(Exception e) {
                                System.out.println("Invalid port number.");
                                System.out.println("Usage is: > java Server [portNumber]");
                                return;
                        }
                case 0:
                        break;
                default:
                        System.out.println("Usage is: > java Server [portNumber]");
                        return;

        }
        // create a server object and start it
        Server server = new Server(portNumber);
        server.start();
}

// One instance of this thread will run for each client
class ClientThread extends Thread {
        // the socket to get messages from client
        Socket socket;
        ObjectInputStream sInput;
        ObjectOutputStream sOutput;
        // my unique id (easier for deconnection)
        int id;
        // the Username of the Client
        String username;
        // message object to recieve message and its type
        ChatMessage cm;
        // timestamp
        String date;

        // Constructor
        ClientThread(Socket socket) {
                // a unique id
                id = ++uniqueId;
                this.socket = socket;
                //Creating both Data Stream
                System.out.println("Thread trying to create Object Input/Output Streams");
                try
                {
                        sOutput = new ObjectOutputStream(socket.getOutputStream());
                        sInput  = new ObjectInputStream(socket.getInputStream());
                        // read the username
                        username = (String) sInput.readObject();
                        broadcast(notif + username + " has joined the chat room." + notif);
                }
                catch (IOException e) {
                        display("Exception creating new Input/output Streams: " + e);
                        return;
                }
                catch (ClassNotFoundException e) {
                }
    date = new Date().toString() + "\n";
        }

        public String getUsername() {
                return username;
        }

        public void setUsername(String username) {
                this.username = username;
        }
```

```
		// infinite loop to read and forward message
		public void run() {
			// to loop until LOGOUT
			boolean keepGoing = true;
			while(keepGoing) {
				// read a String (which is an object)
				try {
					cm = (ChatMessage) sInput.readObject();
				}
				catch (IOException e) {
					display(username + " Exception reading Streams: " + e);
					break;
				}
				catch(ClassNotFoundException e2) {
					break;
				}
				// get the message from the ChatMessage object received
				String message = cm.getMessage();

				// different actions based on type message
				switch(cm.getType()) {

				case ChatMessage.MESSAGE:
					boolean confirmation =  broadcast(username + ": " + message);
					if(confirmation==false){
						String msg = notif + "Sorry. No such user exists." + notif;
						writeMsg(msg);
					}
					break;
				case ChatMessage.LOGOUT:
					display(username + " disconnected with a LOGOUT message.");
					keepGoing = false;
					break;
				case ChatMessage.WHOISIN:
					writeMsg("List of the users connected at " + sdf.format(new Date()) +
"\n");
					// send list of active clients
					for(int i = 0; i < al.size(); ++i) {
						ClientThread ct = al.get(i);
						writeMsg((i+1) + ") " + ct.username + " since " + ct.date);
					}
					break;
				}
			}
			// if out of the loop then disconnected and remove from client list
			remove(id);
			close();
		}

		// close everything
		private void close() {
			try {
				if(sOutput != null) sOutput.close();
			}
			catch(Exception e) {}
			try {
				if(sInput != null) sInput.close();
			}
			catch(Exception e) {};
			try {
				if(socket != null) socket.close();
			}
			catch (Exception e) {}
		}

		// write a String to the Client output stream
		private boolean writeMsg(String msg) {
			// if Client is still connected send the message to it
			if(!socket.isConnected()) {
				close();
				return false;
			}
```

```
                        // write the message to the stream
                        try {
                                sOutput.writeObject(msg);
                        }
                        // if an error occurs, do not abort just inform the user
                        catch(IOException e) {
                                display(notif + "Error sending message to " + username + notif);
                                display(e.toString());
                        }
                        return true;
                }
        }
}
```